Computer Graphics

COMP3421 and COMP9415

Never Stand Still

Introduction

Dr Ali Darejeh
School of Computer Science and Engineering
Term 3, 2023

Course Staff

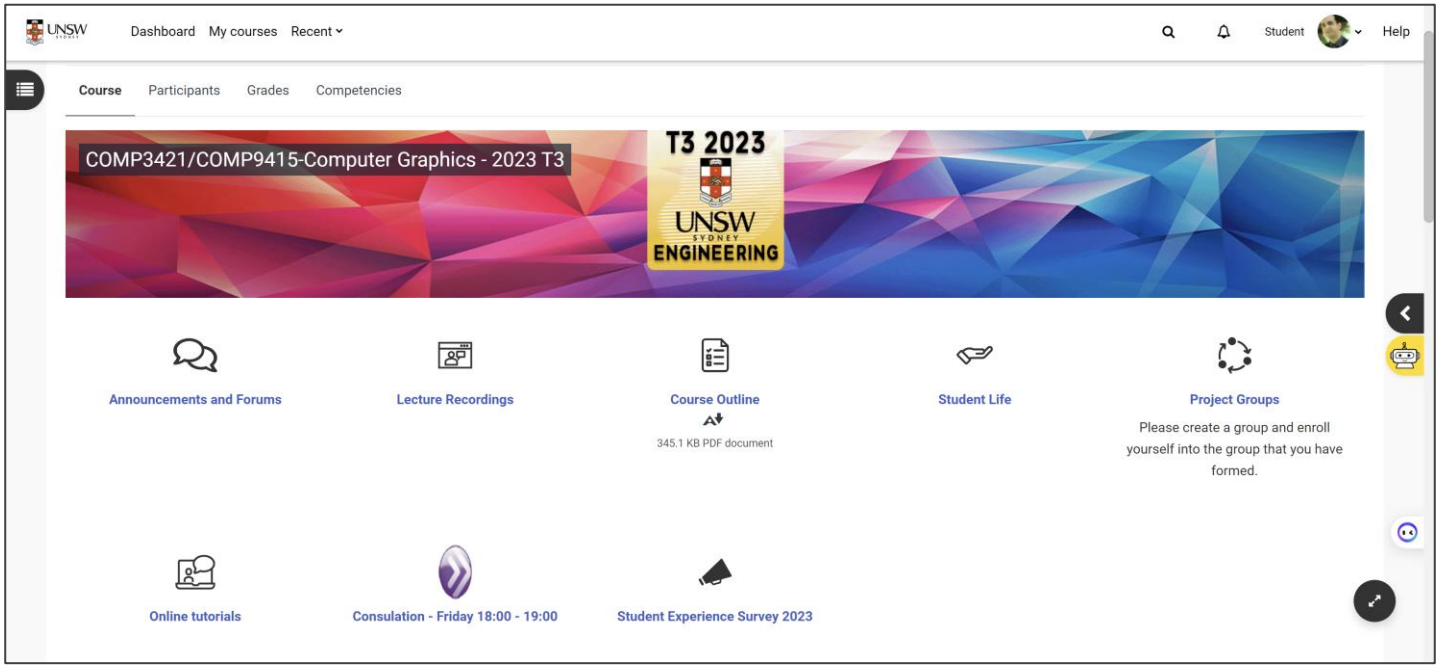
- Lecturer: Dr. Ali Darejeh
- Tutors: Kanit Srihakorth, Guy Chilcott, Ryan Tram, Lauren Huynh, Yi Fu, Xiaohan Ji (Harry), Fengyu Wang (Alan)
- Contact email:
- Ali Darejeh: ali.darejeh@unsw.edu.au

Tutors of each lab

Code	Class	Day/Start Time	Tutor	Email
3956	T09A	Tue 09:00 - 11:00 (Weeks:1-5,7-10)	Lauren	lauren.huynh@student.unsw.edu.au
3957	T11A	Tue 11:00 - 13:00 (Weeks:1-5,7-10)	Lauren	lauren.huynh@student.unsw.edu.au
3958	T13A	Tue 13:00 - 15:00 (Weeks:1-5,7-10)	Yi	yi.fu.1@student.unsw.edu.au
3959	T15A	Tue 15:00 - 17:00 (Weeks:1-5,7-10)	Lauren	lauren.huynh@student.unsw.edu.au
3960	T17A	Tue 17:00 - 19:00 (Weeks:1-5,7-10)	Yi	yi.fu.1@student.unsw.edu.au
3961	T17B	Tue 17:00 - 19:00 (Weeks:1-5,7-10)-Online	Alan	fengyu.wang1@student.unsw.edu.au
3962	T19A	Tue 19:00 - 21:00 (Weeks:1-5,7-10)	Yi	yi.fu.1@student.unsw.edu.au
3963	T19B	Tue 19:00 - 21:00 (Weeks:1-5,7-10)-Online	Alan	fengyu.wang1@student.unsw.edu.au
3964	W09A	Wed 09:00 - 11:00 (Weeks:1-5,7-10)	Guy	guy.chilcott@student.unsw.edu.au
3965	W11A	Wed 11:00 - 13:00 (Weeks:1-5,7-10)	Guy	guy.chilcott@student.unsw.edu.au
3966	W13A	Wed 13:00 - 15:00 (Weeks:1-5,7-10)	Ryan	r.tram@student.unsw.edu.au
3967	W15A	Wed 15:00 - 17:00 (Weeks:1-5,7-10)	Ryan	r.tram@student.unsw.edu.au
3968	W17A	Wed 17:00 - 19:00 (Weeks:1-5,7-10)	Kanit	k.srihakorth@student.unsw.edu.au
3969	W19A	Wed 19:00 - 21:00 (Weeks:1-5,7-10)	Ryan	r.tram@student.unsw.edu.au
3954	H14A	Thu 14:00 - 16:00 (Weeks:1-5,7-10)	Kanit	k.srihakorth@student.unsw.edu.au
3955	H16A	Thu 16:00 - 18:00 (Weeks:1-5,7-10)	Kanit	k.srihakorth@student.unsw.edu.au
11431	H12A	Thu 12:00 - 14:00 (Weeks:1-5,7-10)	Harry	xiaohan.ji@student.unsw.edu.au
11433	F10A	Fri 10:00 - 12:00 (Weeks:1-5,7-10)	Harry	xiaohan.ji@student.unsw.edu.au

Where to ask your questions?

- Moodle is the only website of the course.
- General forum on Moodle for lecture content related questions.
- Project checkpoint forums on Moodle for your questions about your project.

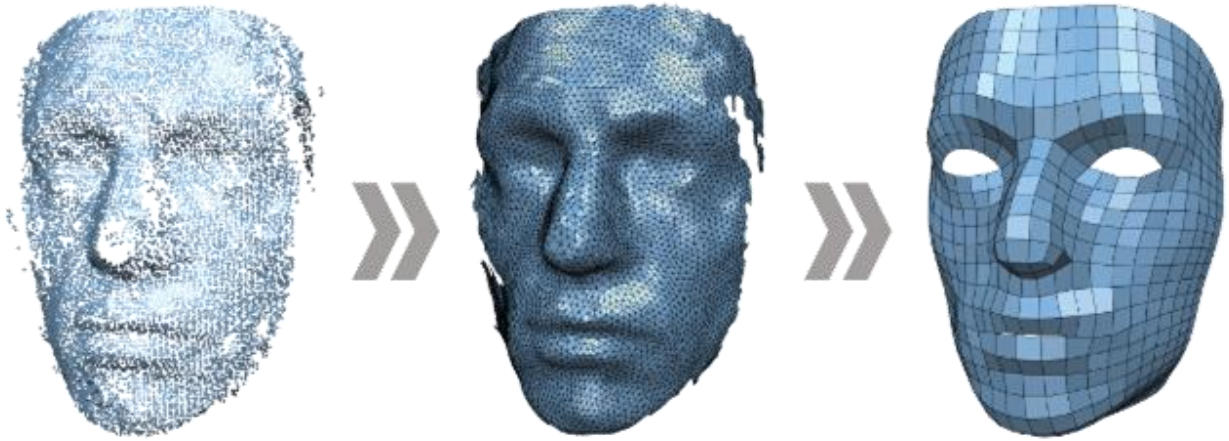


What are we talking today?

- What is computer graphics
- History of computer graphics
- Structure of the course and course syllabus
- Assessment structure
- History of game engines
- Different usages of game engines
- Difference between game engines and graphic rendering API‘s (OpenGL & DirectX)
- History of Video games

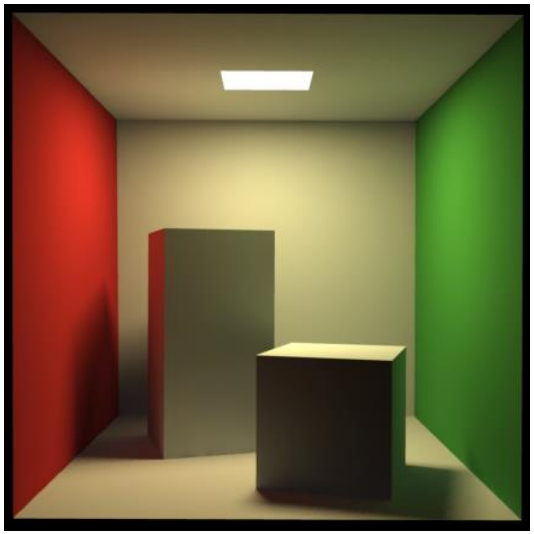
What is computer graphic?

- **Computer graphics** is a sub-field of computer science which studies methods for digitally synthesizing and manipulating visual content.
- **Computer graphics** refers to the study of two-dimensional (2D) and three-dimensional (3D) computer graphics.
- **Computer graphics** is responsible for displaying art and image data effectively and meaningfully to users.



Subfields of computer graphic

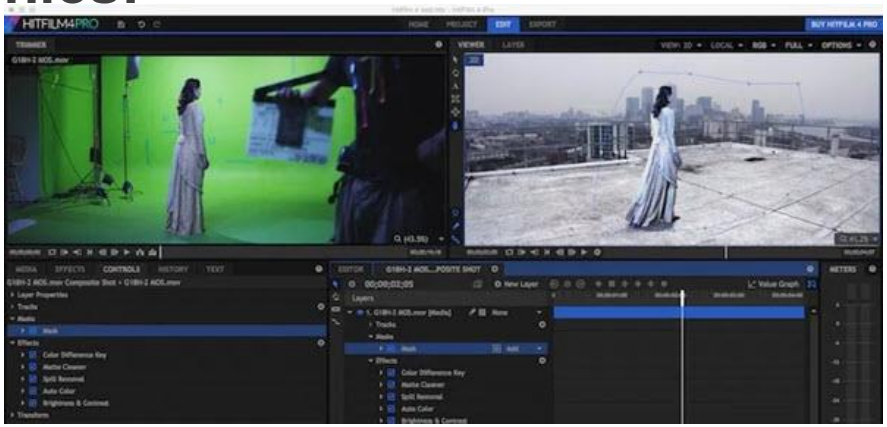
- **Geometry**: ways to represent and process surfaces.
- **Imaging**: image processing or image editing.
- **Animation**: ways to represent and manipulate motion.
- **Rendering**: algorithms to reproduce light transport.



Why study computer graphic?

Different areas of computer graphics:

- Games
- Animations
- Visual effects in films
- Virtual and augmented reality

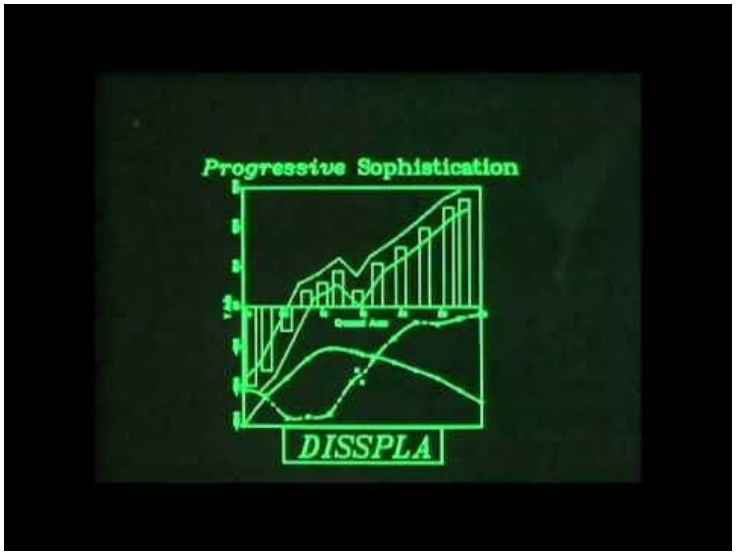


History of Computer Graphics

Computer Gaming (1960 - 1970)

- Drawing 2D graphs
- Emergence of the first video games

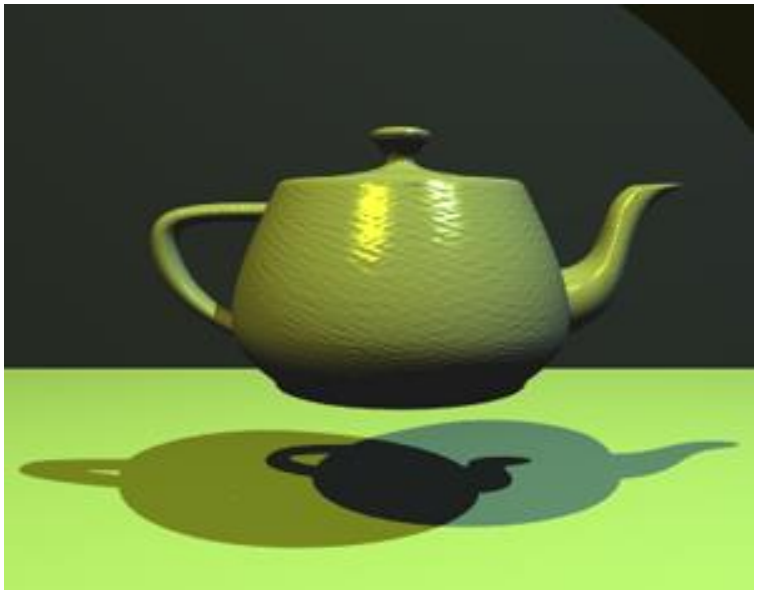
Space war game



History of Computer Graphics

3D modeling (1970-1980)

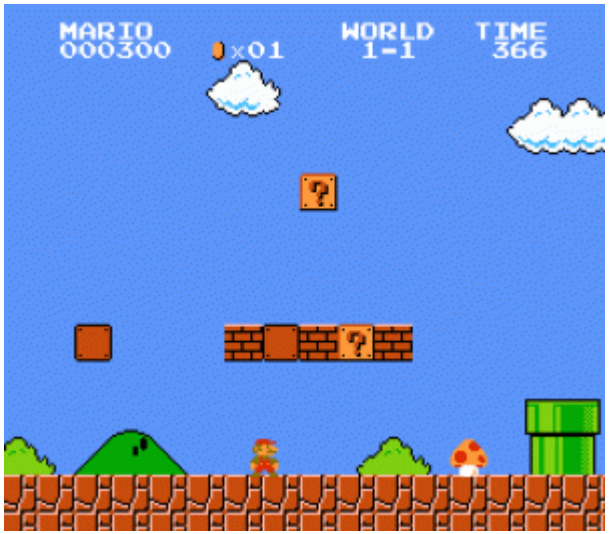
- Emergence of the 3D Core Graphics System

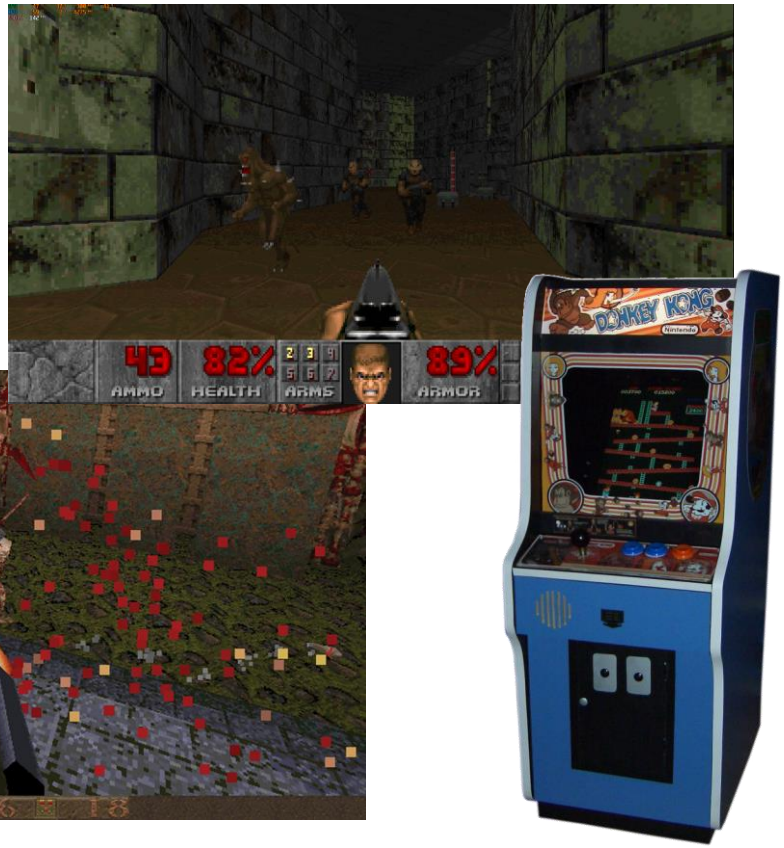


History of Computer Graphics

Computer graphics to develop video games (1980 - 1995)

- 2D games, Arcade machines
- 3D games (Doom, Quake)

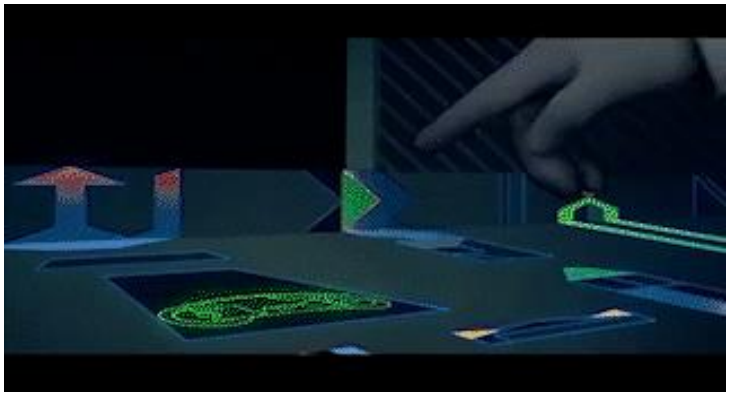


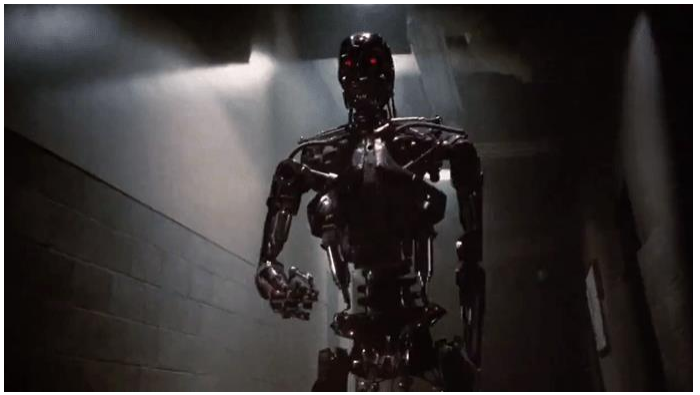


History of Computer Graphics

Films were involved (1980-1995)

Tron (1982), The Terminator (1984), Jurassic Park (1993), Toy Story (1995)

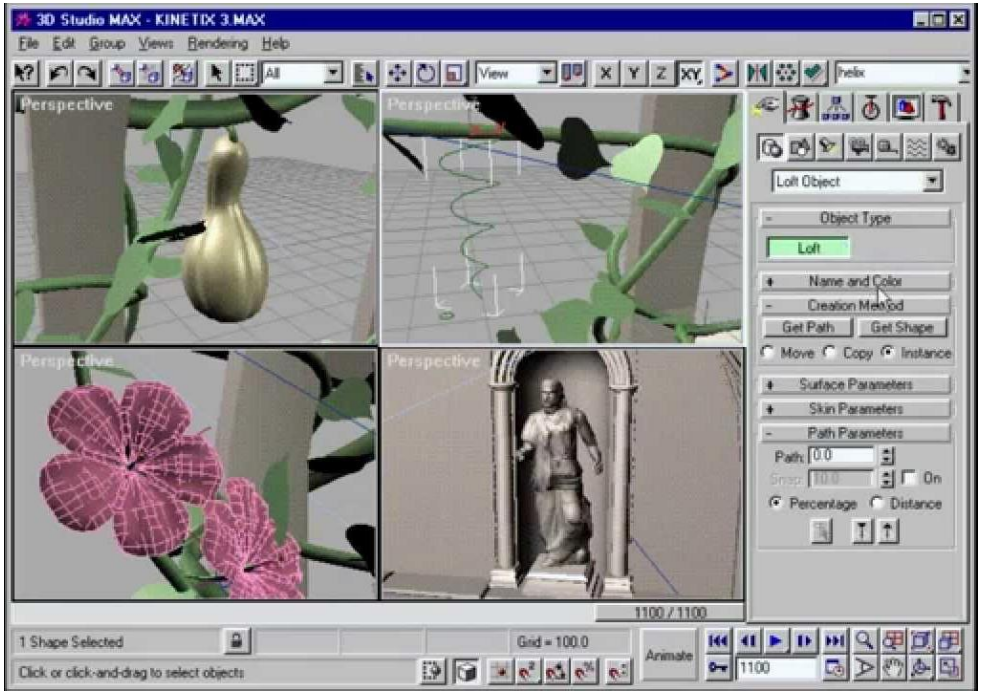




History of Computer Graphics

3D modeling applications (1990-2000)

Emergence of 3D modeling applications and an impressive rise in the quality of computer-generated graphics and 3D games.



UNSW AUSTRALIA

History of Computer Graphics

High quality 3D games and visual effects (2000-2010)

Video games and cinema had spread the reach of computer graphics to the mainstream by the late 1990s and continued to do so in the 2000s.

History of Computer Graphics

Realistic 3D games and virtual reality (2010- present)

Real-time graphics on a suitably high-end system simulate photorealism.

Forza Horizon 5 game

Shooting game in VR

What's in the Course?

Course Overview

- The Course Outline: https://webcms3.cse.unsw.edu.au/COMP3421/22T3/outline
- Lectures
  - Teaching fundamentals of computer graphics by using practical projects in game engines
- Tutorials
  - Working on your proposed project and getting consultation from your tutor

Lectures and teaching method

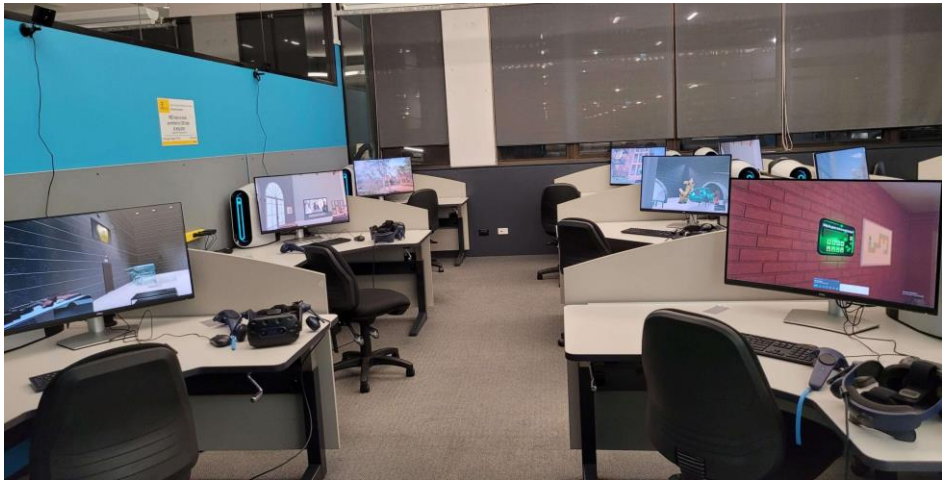
- Teaching using practical projects
- Lecture activities to practice lecture content
- Lectures (In-person)
- Time
  - Monday 6-8pm (Rex Vowels Theatre (K-F17-LG3))
  - Thursday 6-8pm (Physics Theatre (K-K14-19))
  - They are not repetitive.
- Recordings will be made available on Echo360 via Moodle.

Tutorials

- Weekly meeting with your tutor
- Working on your proposed project and getting consultation from your tutor
- Project progress updates
- Attendance to labs is Compulsory (you should attend at least 80% of the labs)
- **In-person tutorials:** VR lab (Moog Lab), Ground floor, CSE building (K17).
- http://vrlab.au/
- **Online tutorials:** Blackboard Collaborate on Moodle.

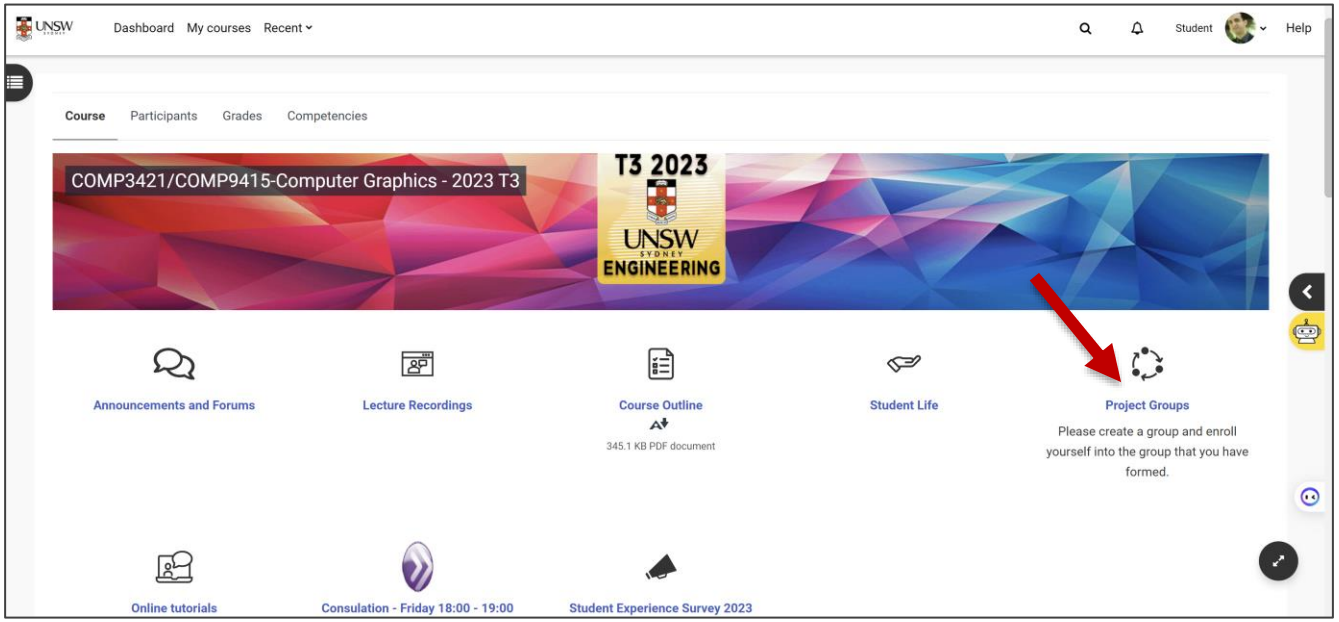
Tutorials

- You need to have:
  - Steam account
  - Epics game account
  - Unity account
- After logging to the lab's computers, log into Steam, Unreal Engine launcher, and Unity launcher using your own accounts.
- Every two students can have one computer and one virtual reality headset in the lab.



Projects group

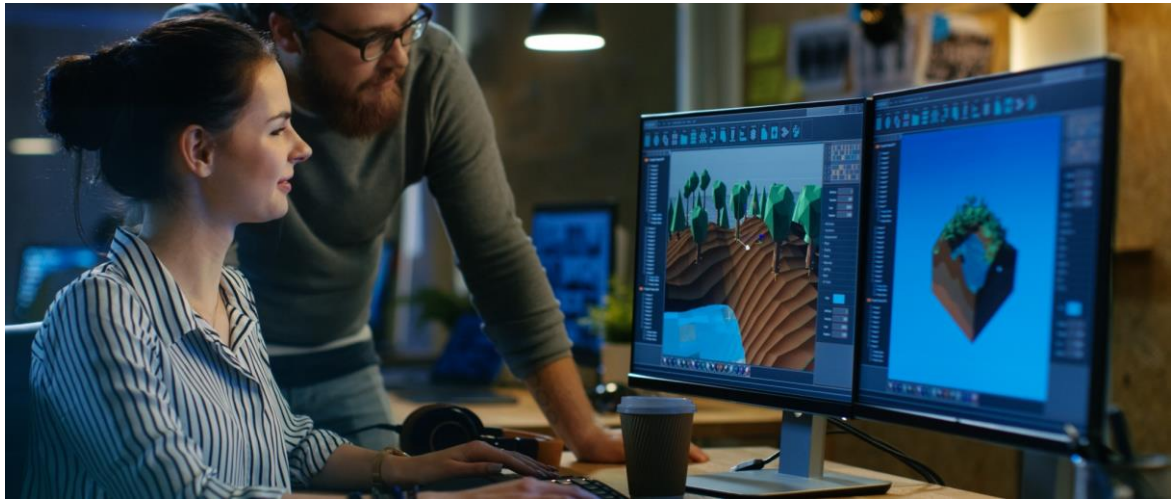
- Students work in teams of ideally four to five members.
- Choose a name for your team.
- Choose a group admin.
- Add your team into Moodle (Just the team admin should create the group and the other members should join the group).



The purpose of this course

This course will:

- Make you ready to work in industry in the areas of developing graphical interactive environments and video games.
- Teach you practical side of computer graphics and the necessary theoretical background such as geometry.



Topics we're covering

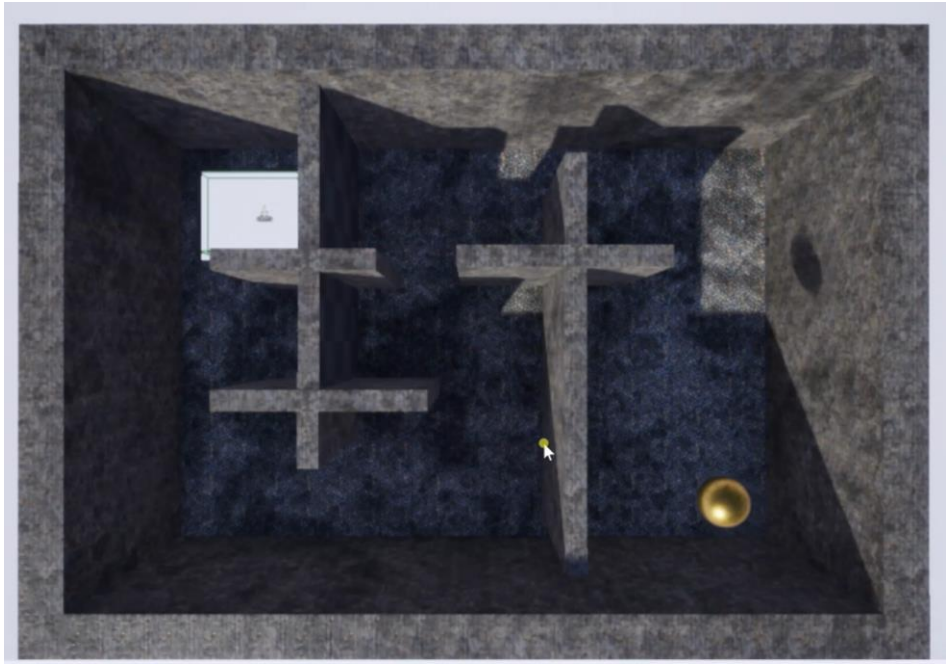
- How to develop 2D, 3D and virtual reality based graphical objects and environments with the use of game engines (Unreal Engine and Unity).
- Computer graphic concepts including lighting, reflection, shader, static meshes, projectiles, 2D Transforms, 3D Transforms, surface, texture maps, materials, cameras, objects physical behaviour, collision detection, hierarchical modelling of objects and rendering.

Will you learn low or high level computer graphics in this course?

- High level computer graphics in addition to the fundamental knowledge related to low level computer graphics.
- Because you want to develop a real-world project.
- The lecture time doesn't allow us to cover both low and high level computer graphic concepts.
- Some necessary Geometric concepts will be covered

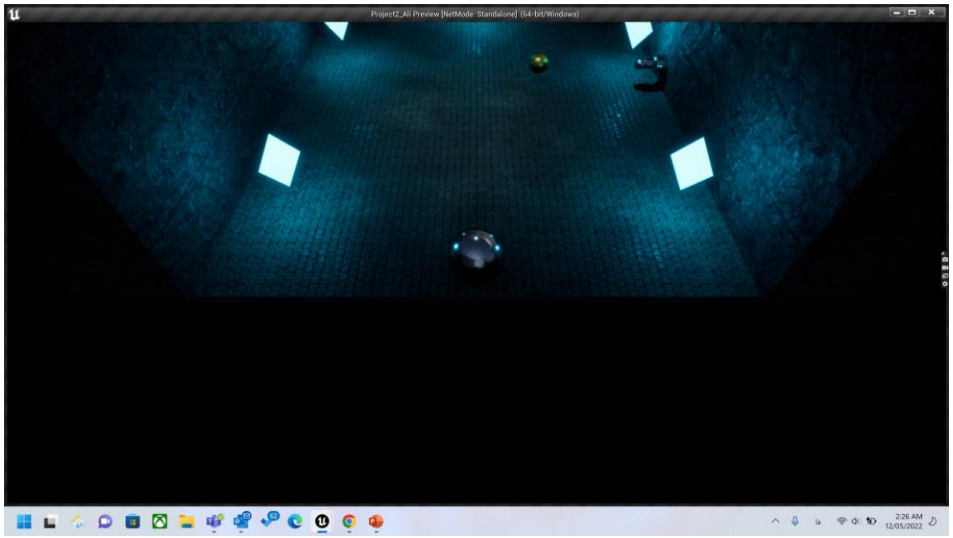
Lectures content (Project 1)

- Player pawn and transform tools in 3D
- Mapping controllers to move objects in 3D
- Introduction to material concept
- Introduction to geometry brushes in 3D
- Introduction to static meshes
- Introduction to collision detection and physics in 3D
- Introduction to the trigger box 3D
- Introduction to rendering
- Changing surface transform by keyboard and mouse



Lectures content (Project 2)

- Geometry brushes in 3D
- Introduction to lights and shadows
- Player pawn class and camera
  Mapping controllers to rotate objects in 3D
- Advanced concepts of camera
- Material design
- Advanced collision detection and physics in 3D
- Animating objects
- Creating projectile in 3D
- Importing prebuilt graphical elements (FBX files)
- Applying damage to the player pawn
- Creating conditional trigger box

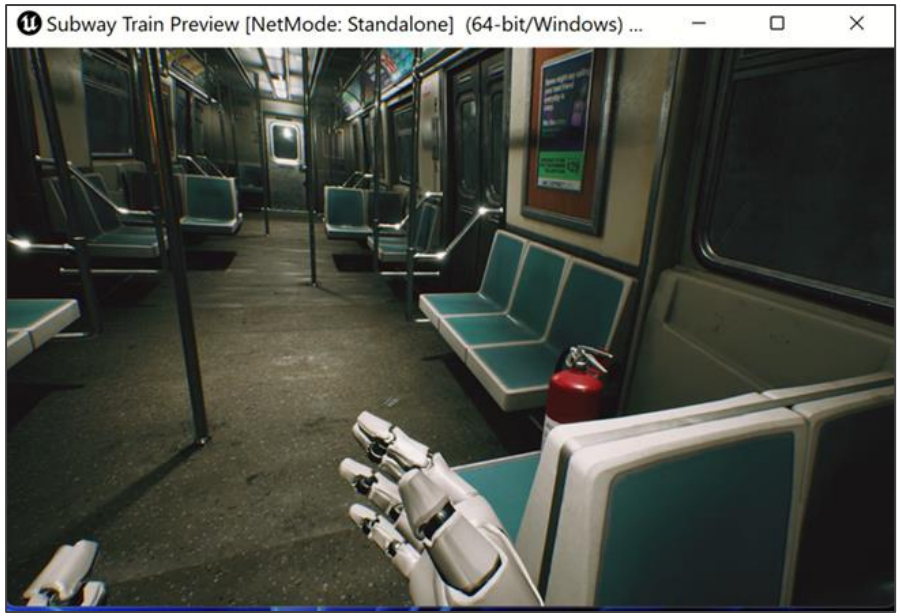


Lectures content (Project 3)

- Landscape tools
- Texture layers
- Advanced concepts in landscape brushes
- Applying a texture to a landscape
- Add foliage
- Advanced concepts in material design

Lectures content (Project 4)

- Introduction to virtual reality
- Rendering VR scenes
- Mapping virtual reality controllers
- Creating VR player pawn
- Developing VR hand
- Grabbing and moving objects in VR
- Movement in VR
- Snap turn in VR
- Teleportation in VR



Lectures content (Project 5)

- 2D Objects and transform
- Player pawn 2D
- Introduction to C# programming
  Mapping controllers in 2D
- Prefab objects and projectiles in 2D
- Physics and collision detection 2D
- Introduction to trigger box 2D
- Applying damaging and destroying objects 2D
- Spawn manager
- Adding user interface elements
- Animating objects in 2D
- Rendering 2D sprites

Course learning outcome

- After completing this course, students should be able to:
- Work with the game engines in order to design 2D, 3D and virtual-reality-based graphical environment and objects including lines, curves, surfaces, geometrical shapes, etc.
- Work with the computer graphic elements, such as lighting, shadow, surface, texture map, physical behaviour of objects and camera.
- Render scenes on a range of platforms.

This course is appropriate for you if:

- You’re interested in designing interactive graphical environments and video games.
- You're interested to work in a project group and design your own idea.
- You know the fundamentals of programming.
- You want to learn game engines (No prior game engine experience is required).
- You want to learn high level computer graphics programming.

This course is not appropriate for you if:

- You don't like project-based courses and you prefer doing assignments and final exam.
- You're not interested in designing interactive graphical environments or video games.
- You want to learn only low-level computer graphics, mathematics, or OpenGL.
- You want to learn general programming.
- You have an advance knowledge of game engines and you only want to learn graphics APIs.

What should you do in this Course? (Assessments)

- This is a project-based course. Students work in teams of ideally five (5) members.
- The project has two phases:
  - **Phase 1:** Define, implement and evaluate a choice of either a 2D or a 3D interactive environment.
  - **Phase 2:** either transform your existing project into a VR-based experience or incorporate a mini-system into your current project for added functionality and engagement.
  - Project teams meet weekly starting from Week 1 with project mentors to report on the progress of the project.

Project milestones

Weeks	Deliverables	Mark
Week 4	Proposal	10%
Week 8	Project presentation (Phase 1: 2D or 3D system)	10%
Week 10	Project presentation (Phase 2: Mini system or VR system)	10%
Week 11	Project report	10%
Week 11	Project quality	60%

Best projects award

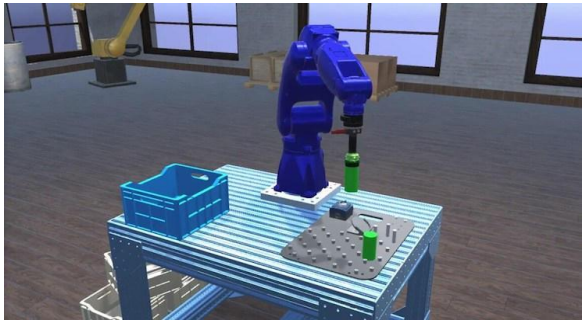
- Ten groups will be selected
- An award will be given to them

What project should you work on

- You should design and develop a graphical interactive system that has a specific purpose.
- Use either Unreal engine or Unity engine to develop:
  - A simulated Environment or
  - A learning platform or
  - A video game

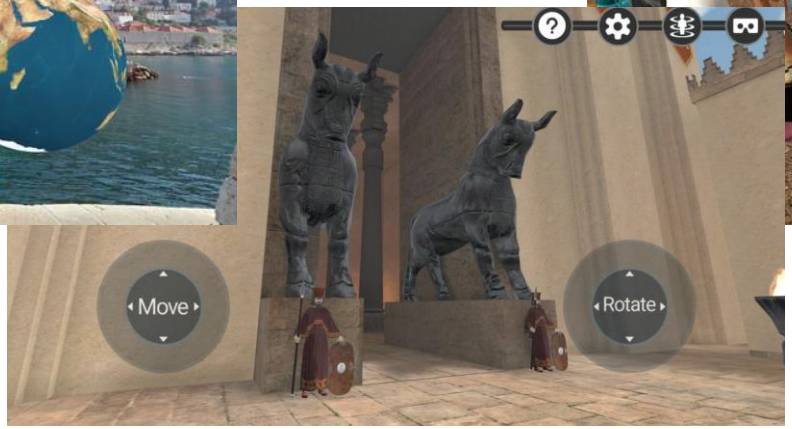
Potential projects that you can work on

A graphical interactive environment such as a laboratory, Medical related topics including human body organs and surgery room, an industrial production line with manufacturing robots, robotics, industrial Machineries, factory,…



Potential projects that you can work on

- An E-learning platform for teaching a specific topic with interactive elements.
- Teaching history, geography, geology, aerospace, medical science, physics, and chemistry.

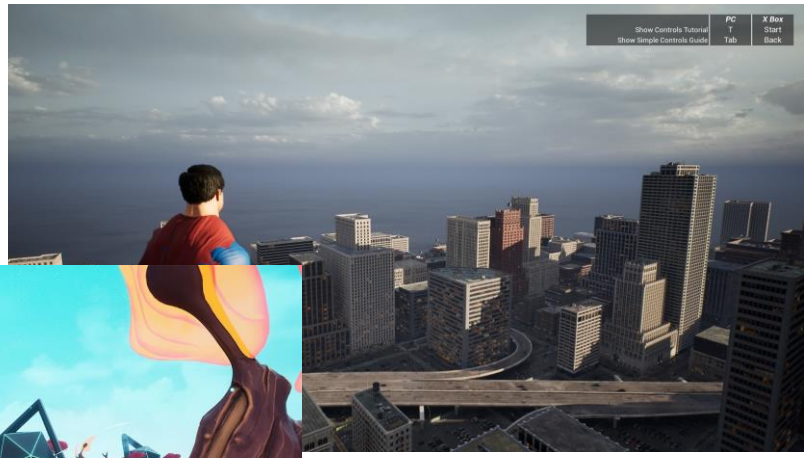


Potential projects that you can work on

- A video game
- It can be either a first person or a third person.

Characteristics of your project

- Your project should have a theme and a purpose
- You should design a graphical environment such as a landscape, a city, a building, a laboratory, a class, a shopping center, an amusement park, a water park, a museum, a restaurant, a castle, a house, a battlefield, or a fictional environment.



Characteristics of your project

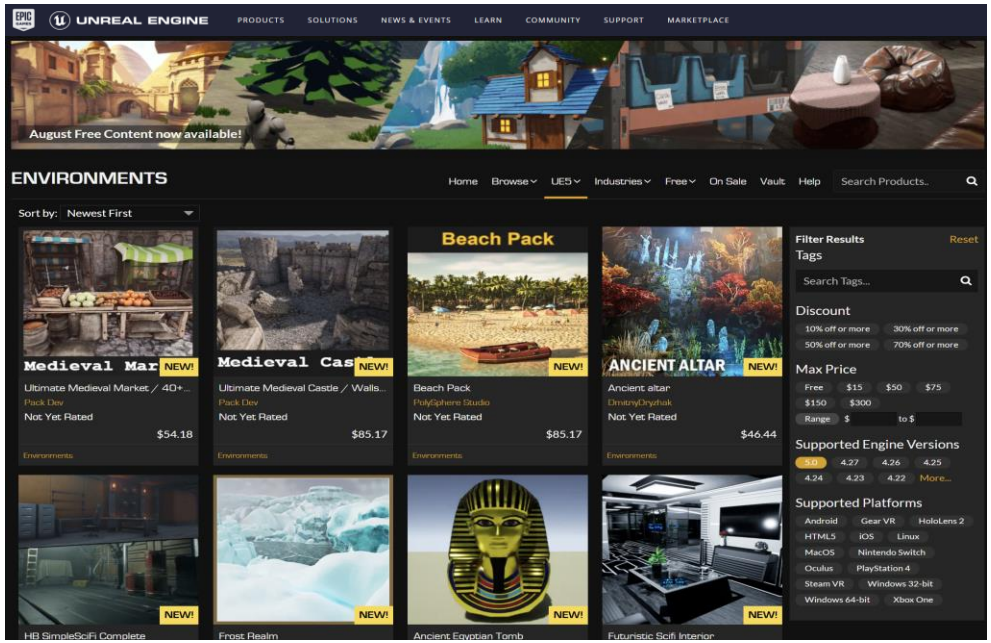
- Based on the environment that you design, there should be related objects in the environment.
- For example, if you're designing a chemistry laboratory, there should be different tools and materials that can be touched and moved by the player.

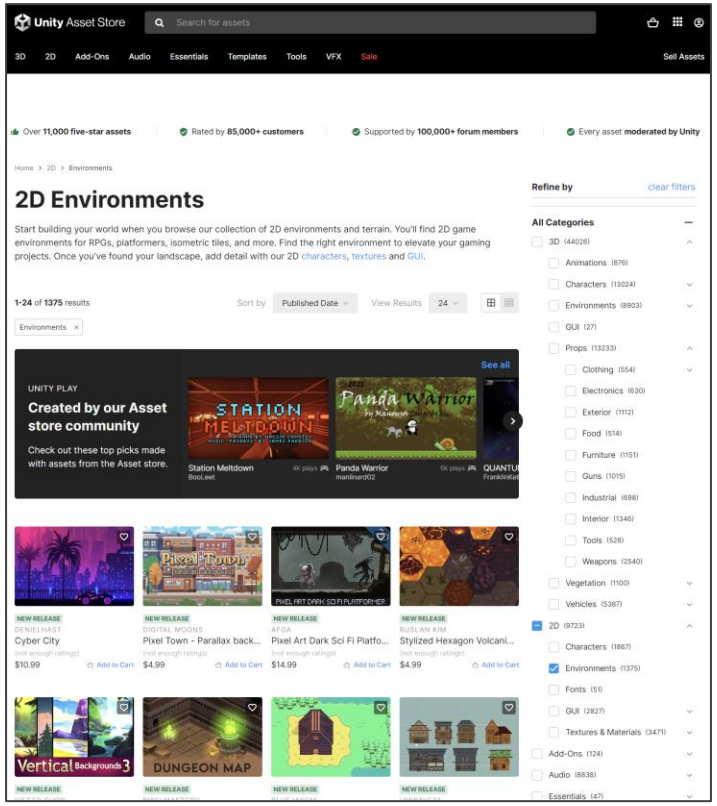
Characteristics of your project

- The system that you design doesn't need to have:
  - Different levels
  - Artificial intelligence such as AI enemy
  - Sound effects
- It is not necessary to design your game objects from scratch in 3D applications such as Blender or 3D Max.
- This course is not a graphic design course, it’s a computer graphics course.

Characteristics of your project

- You can use Unity and Unreal Engine asset stores to download the desired environment and objects and then customise them.





https://www.unrealengine.com/marketplace/en-US/store

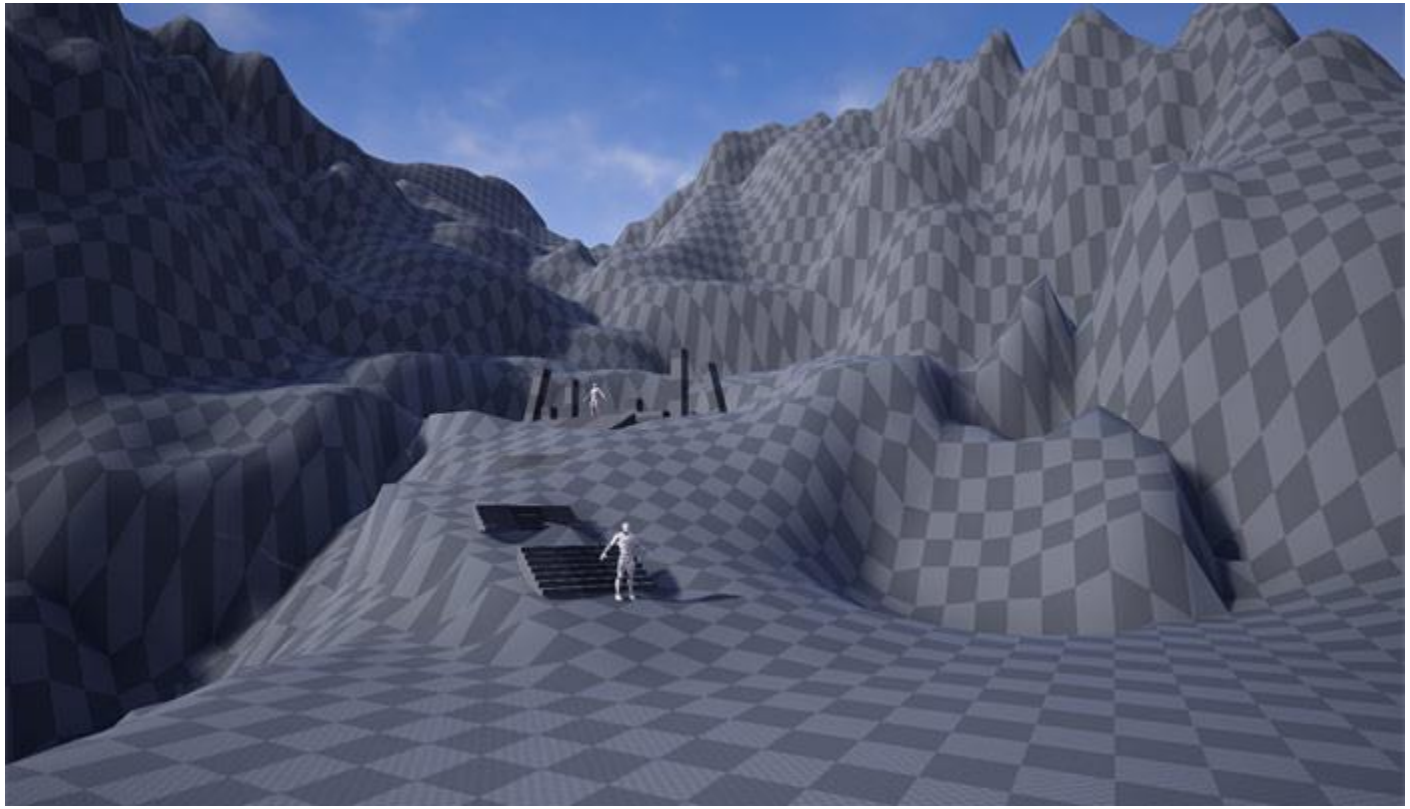
https://assetstore.unity.com/

Characteristics of your project

- You CANNOT just download and present an asset from the asset store, it should be fully customised.
- You don't need to customize simple objects such as fruits, foods, or tools.
- Downloading a project from the Internet or using a pre-existing asset without customization would be considered plagiarism.

Characteristics of your project

- You should design your own environment using Geometry Brushes or Landscape tools, then add the assets that you downloaded from the asset store and customise them.



Characteristics of your project

In order to design and develop your environment, you should consider the items below:

- Use an appropriate lighting technique.
- Use an appropriate camera technique. It can be either a first person or a third person.
- Design your own material and apply them to objects.
- Objects should have physics and collision detection.
- The system should have a backend either by **C++** or **Blueprints** in Unreal Engine or **C#** in Unity.

Characteristics of your project

- You should have a character (the player) in your environment that is controllable by keyboard, mouse or gamepad.
- The character can be a human like character, an animal, a fictional character, a vehicle, or even a geometrical shape based on the theme of your project.
- The character should be able to have physical interaction with the other objects in the environment such as colliding with them, grabbing them, or destroying them.
- The environmental objects should be able to have physical interaction with the player character such as colliding with the player, attacking the player or destroying the player.

Phase 2 (Option 1: Convert your project into a virtual reality-based system)

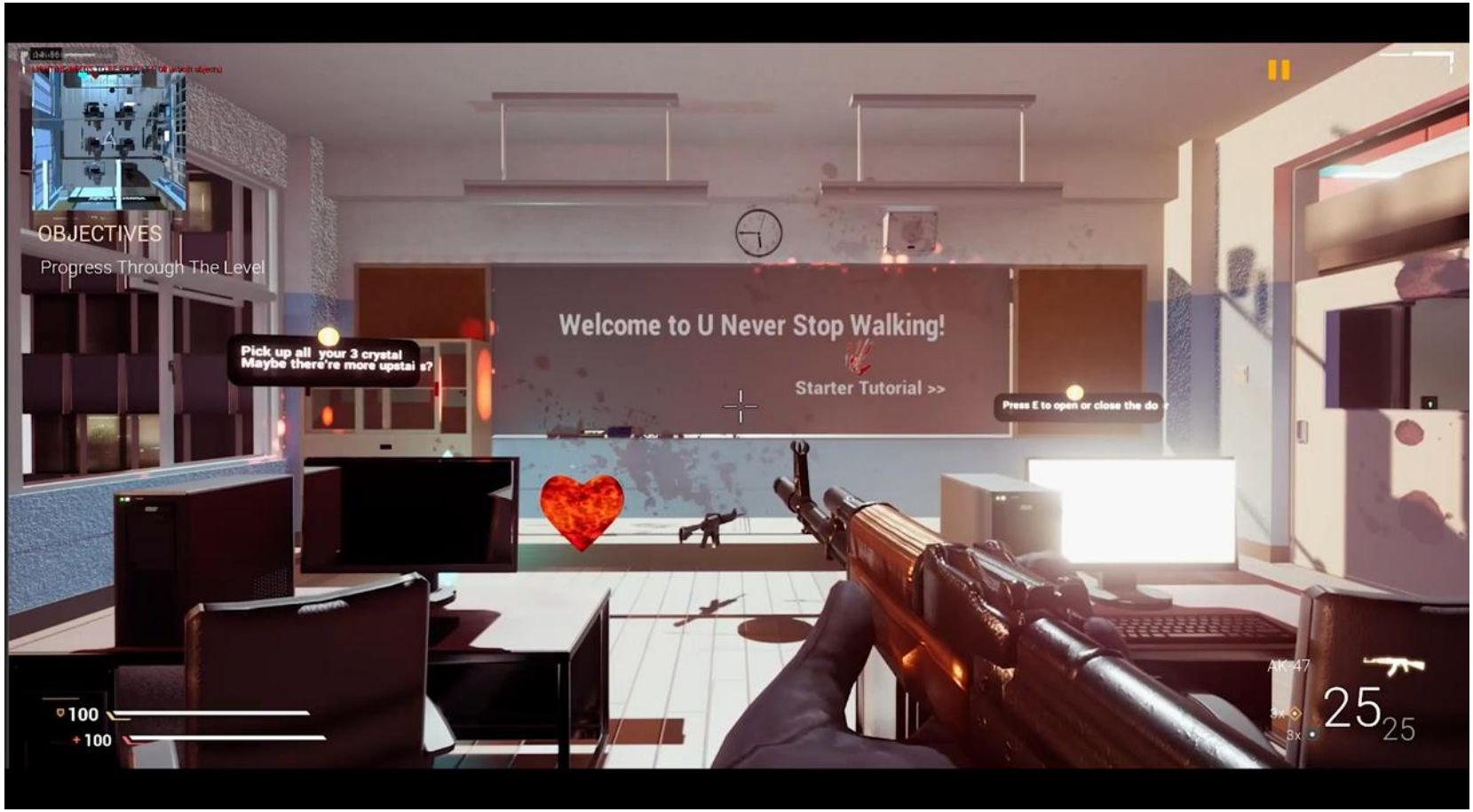
- Convert your project into a virtual reality based system.
- Add VR player
- Map VR controller including both HTC VIVE and Meta Quest Headsets to control the VR player.
- Add VR hand and enable users to grab and move objects with the use of the VR hand.
- The system that you design should work on the VR headsets that we have in the lab (HTC VIVE Pro).

Phase 2 (Option 2: Develop a mini system or a game related to your main project)

- If your project is a video game, consider adding a mini-game within your main game. For example, you could include arcade machines that allow players to engage in simple 2D games, similar to what is seen in GTA.
- If your project is a simulated environment, such as a lab, consider adding functional equipment in addition to the standard equipment already present in your lab.

Sample projects

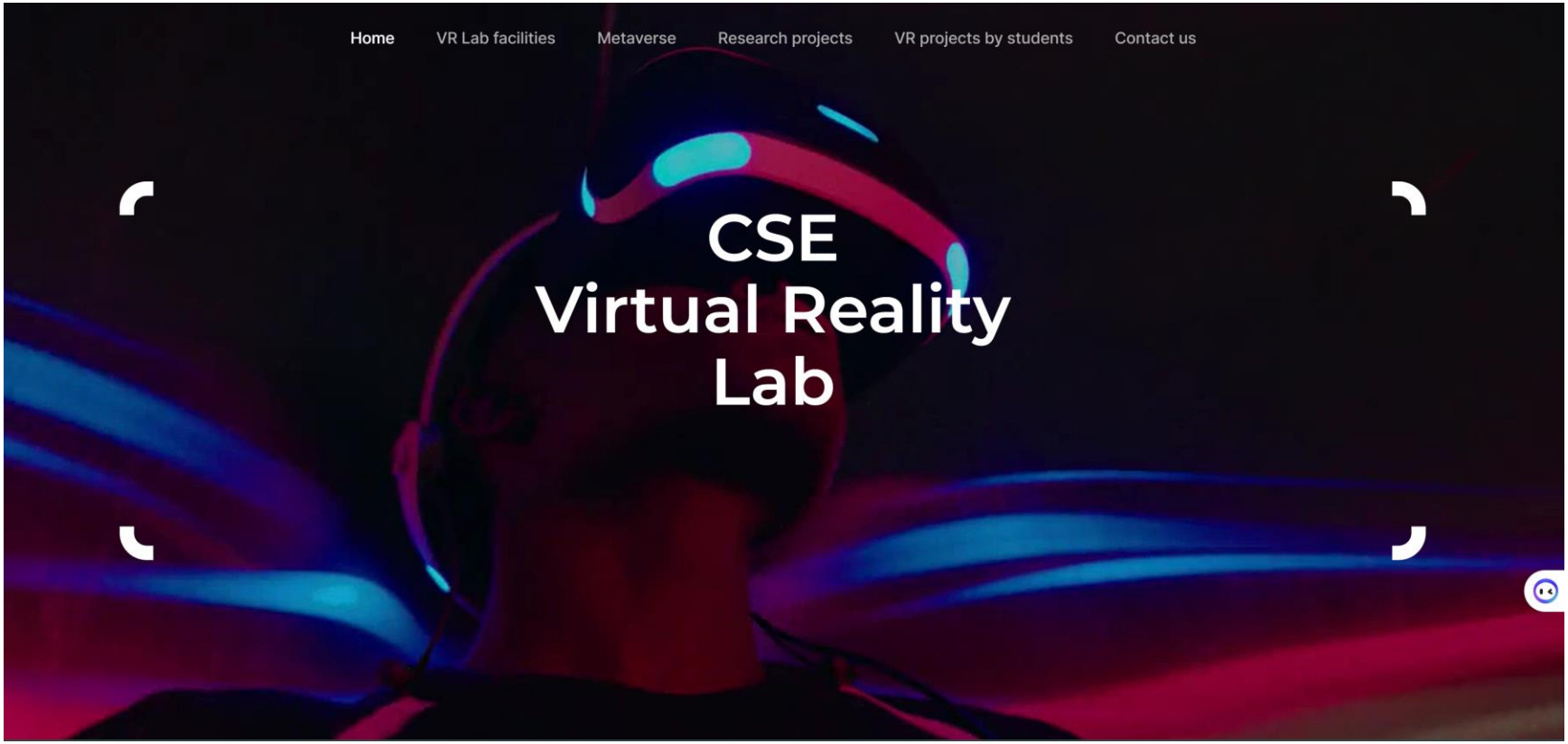
Sample projects



Sample projects

Sample projects

The VR lab website for more sample projects



http://vrlab.au/

What is a game engine?

A game engine is a visual software development tool used to develop:

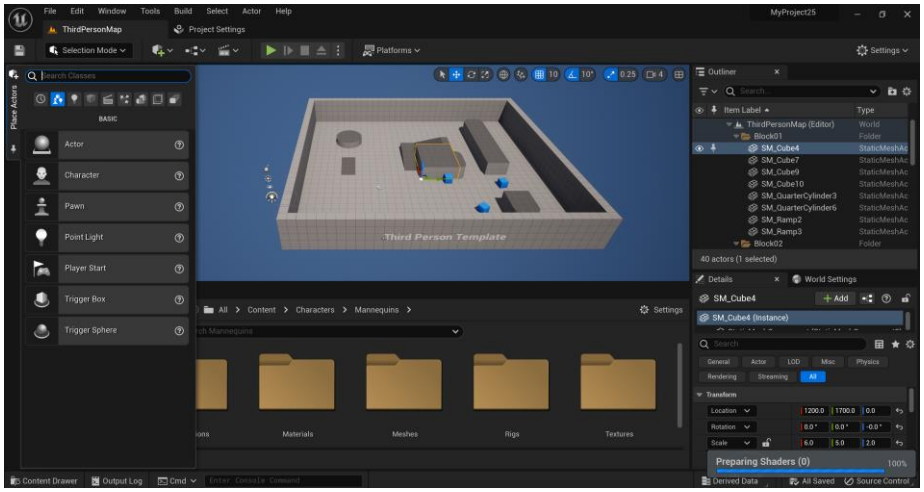
- Video games
- Animations
- Highly interactive graphical environment

What is a game engine?

Game engines provide:

- Level design tools
  - Lighting
  - Material editor
  - Drawing tools
- Cinematic (Animation tools)
- Physics and collision engines
- Artificial intelligence
- Audio engines
- Scene graph (Objects hierarchy)
- Rendering engine for 2D or 3D graphics
- Multiplayer features
- Programming libraries for designing video games.

Unity game engine

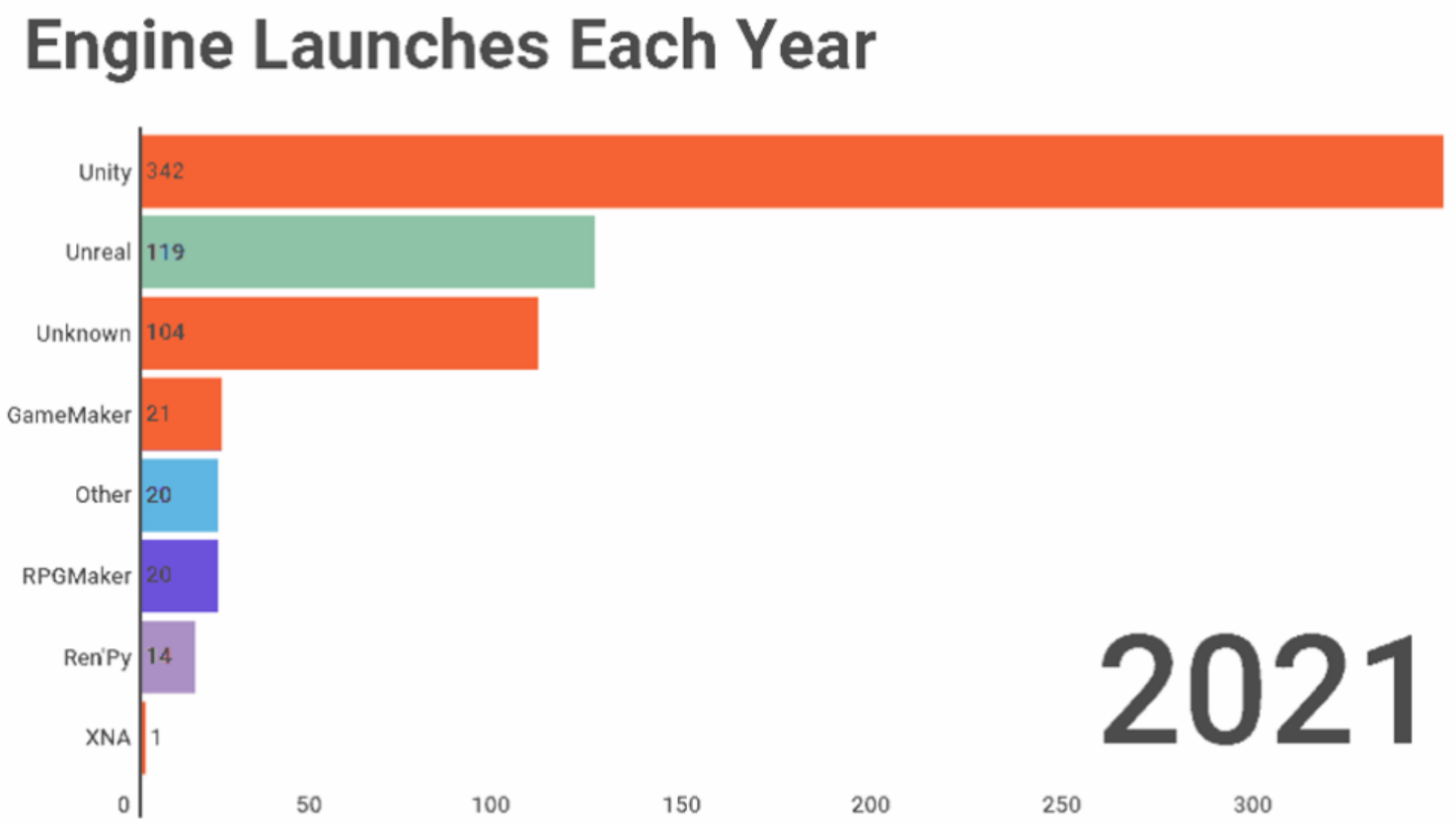


Unreal game engine

Are Unreal Engine and Unity the only game engines

- Amazon Lumberyard
- Cocos2d
- CryEngine
- GameMaker Studio
- Godot
- Honorary Mentions
- Open3D Amazon
- Phaser
- Unity
- Unreal

Comparison between different game engines



The most popular game engines

https://www.gamedeveloper.com/business/game-engines-on-steam-the-definitive-breakdown

Comparison between different game engines

	Installation & Ownership	2D/3D	Ease of Use	Integration & Compatibility	VR Support
Unreal Engine	***	Both	***	******	Yes
Amazon Lumberyard	***	3D Only	*****	***	Yes
CryENGINE	***	Both	***	******	Yes
Unity	***	Both	***	*****	Yes
GameMaker: Studio	******	2D Only	*****	***	No
Godot	*****	Both	*****	***	No
Cocos2d	*****	2D Only	*****	***	No

Why Unity Engine and Unreal Engine?

- From realistic physics in a game environment to landscapes and structures, these engines offer everything a developer needs to design advanced mobile, PC, and console games.
- Unity Technologies, known for its focus on mobile gaming especially 2D games.
- Unreal Engine, known for its PC and console game engine especially 3D games.

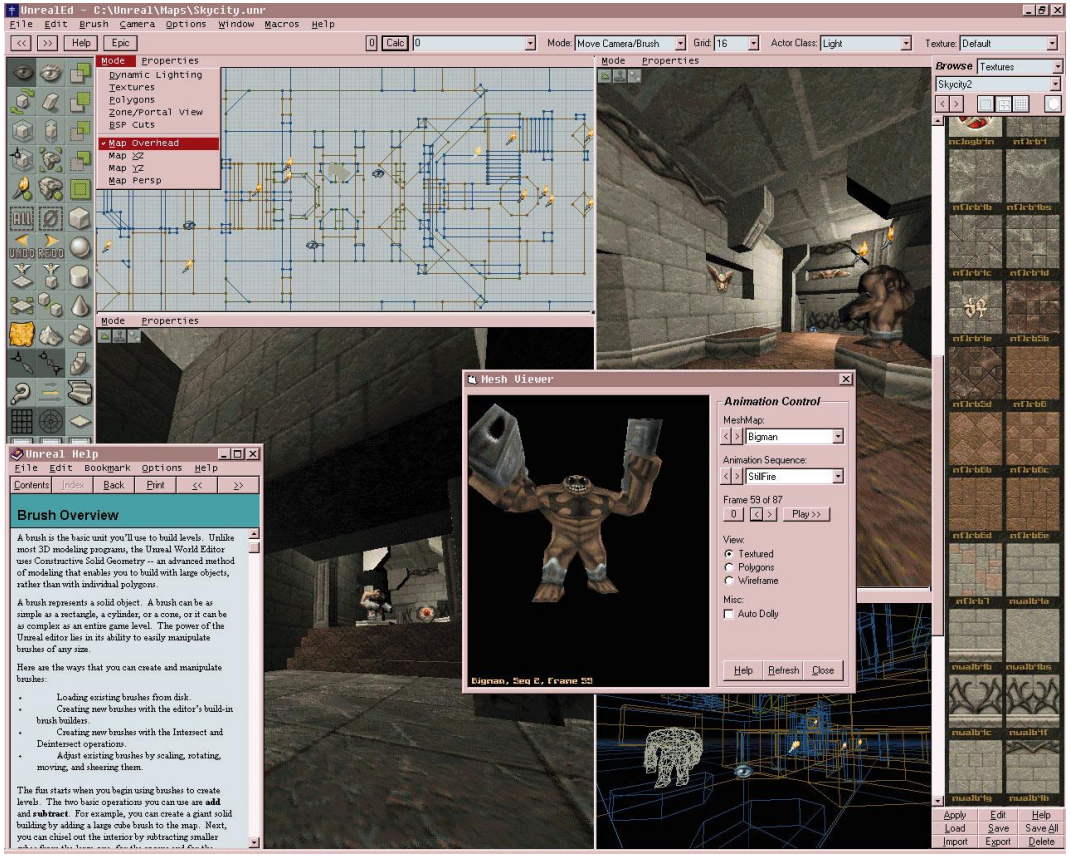
Why Unity Engine and Unreal Engine?

- The Unity Engine and the Unreal Engine currently being the two most popular choices for game developers (CB insights, 2018).
- The $120B gaming industry is being built on the backs of these two engines (CB insights, 2018).
- Pokémon Go (made with Unity Engine) and Fortnite (made with Unreal Engine).

Soirce: https://www.cbinsights.com/research/game-engines-growth-expert-intelligence/

Unreal engine

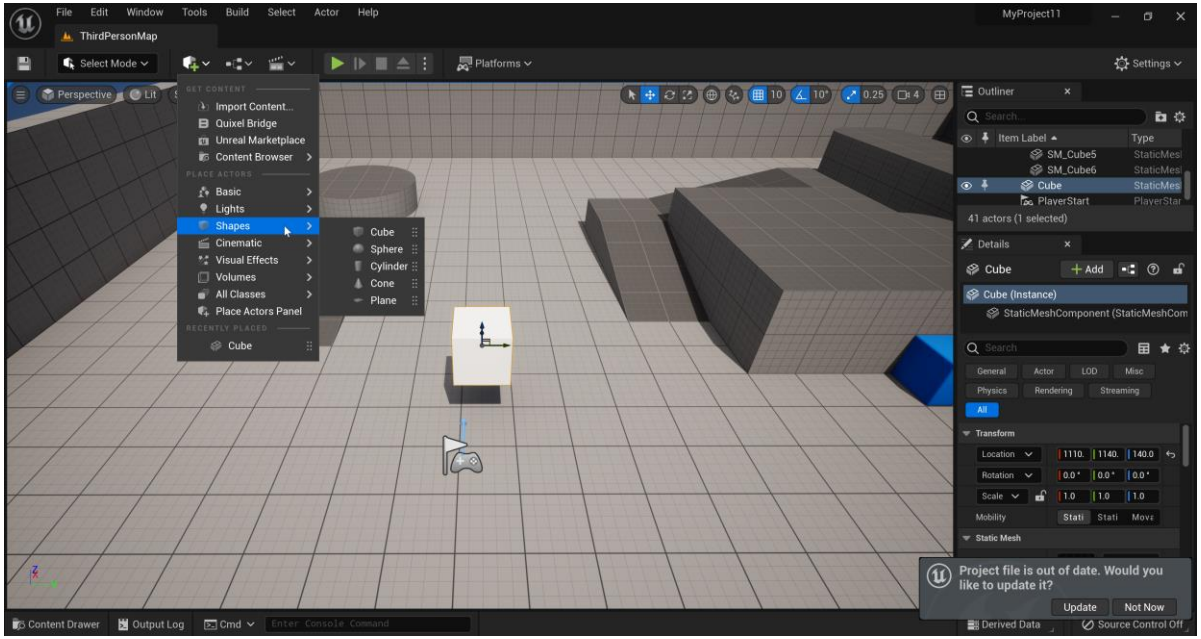
- Unreal Engine developed by Epic Games, was introduced in 1998 as one of the first publicly available gaming engines.
- It grew out of Epic's Unreal video game series, which was one of the first successful online multiplayer games.
- After publishing the game, Epic chose to share the engine and tools used to develop and support it, and the Unreal Engine was born.



Unreal Engine Version 1

Unreal engine

- Unreal Engine provides world's most advanced real-time 3D creation tool sets.
- The company has recently optimized its engine for AR/VR gaming, while partnering with established AR/VR organizations like Meta (Facebook).
- Latest version: Unreal Engine 5.3



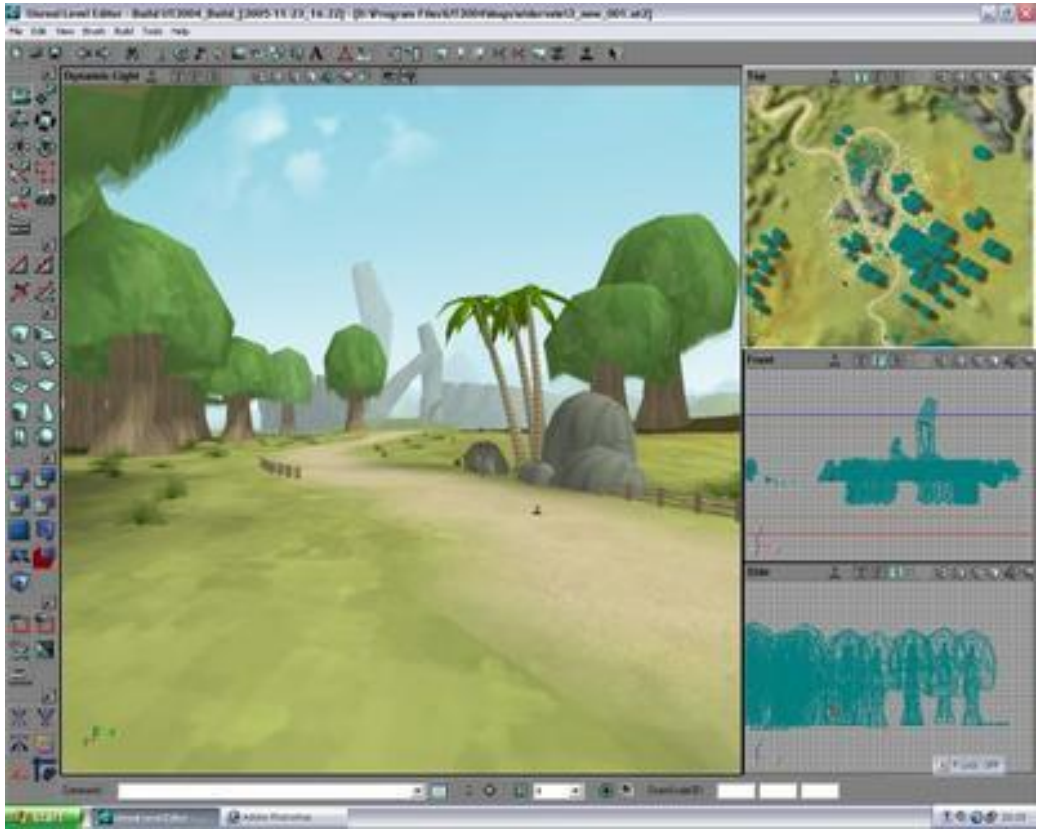
UNREAL ENGINE

UNSW AUSTRALIA

Famous games that are developed by Unreal engine

Unity engine

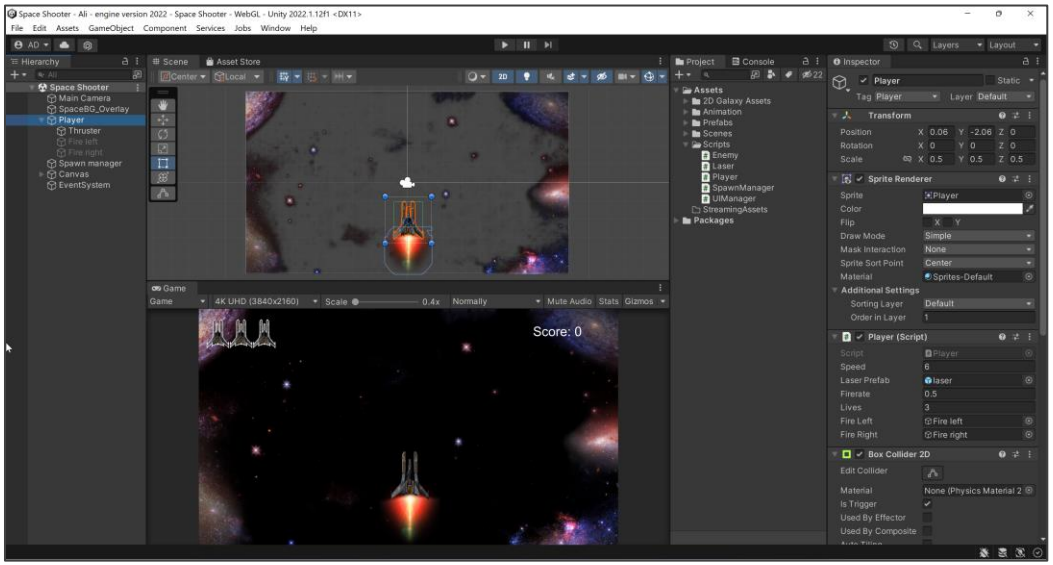
- Unity is a cross-platform game engine developed by Unity Technologies, first released in June 2005 as a Mac OS X game engine.
- The engine has extended to support a variety of desktop, mobile, console and virtual reality platforms.
- It is particularly popular for iOS and Android mobile game development.



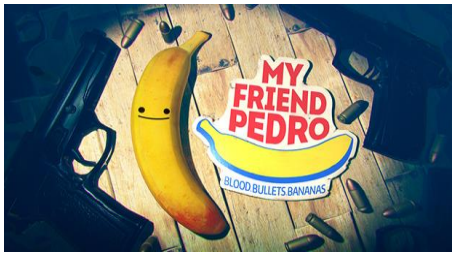
Unity Engine Version 1

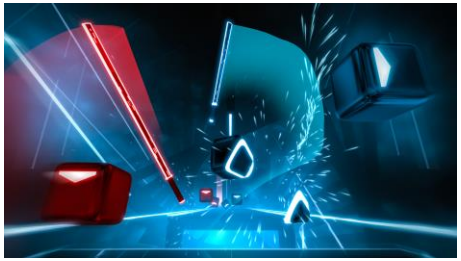
Unity engine

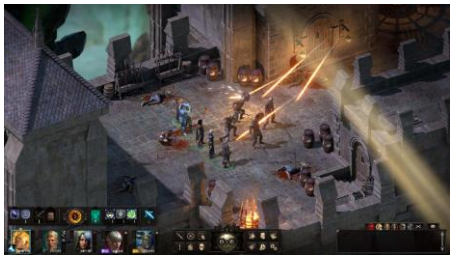
- In 2016, over 30% of the top 1000 mobile games were built using Unity, which reported having 45% of the entire global game engine market.
- Unity will be an important contributor to future growth mobile gaming industry, providing the digital infrastructure necessary to take the mobile gaming segment $100B+ in the next few years.
- Many of the mobile games published by incumbents like Tencent and NetEase leverage the Unity Engine.
- Latest version 2022.3



Famous games that are developed by Unity engine







UNSW AUSTRALIA

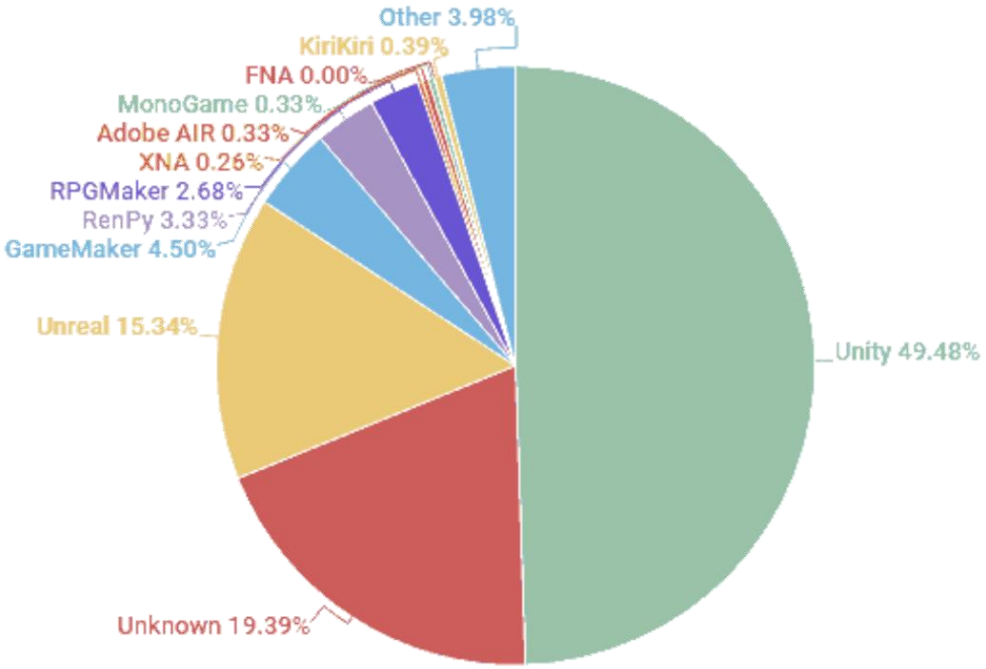
Unity versus Unreal Engine

	Unity	Unreal
Graphics	Physically-Based Rendering, Global Illumination, Volumetric lights after a plugin installed, Post Processing	Physically-Based Rendering, Global Illumination, Volumetric lights out of the box, Post Processing, Material Editor
Unique Features	Rich 2D support	AI, Network Support
Target Audience	Mostly indies, coders	AAA-game studios, indies, artists
Coding	C#, Prefab, Bolt	C++, Blueprints
Community	More than 200k members on the official subreddit	About 100k members on the official subreddit
Performance	Does not scale well, unlike Unreal Engine	Has support for distributed execution (Incredibuild)

https://www.gamecareerguide.com/features/2043/unreal_vs_unity_3d_choosing_the_.php?page=2

Why to learn game engines

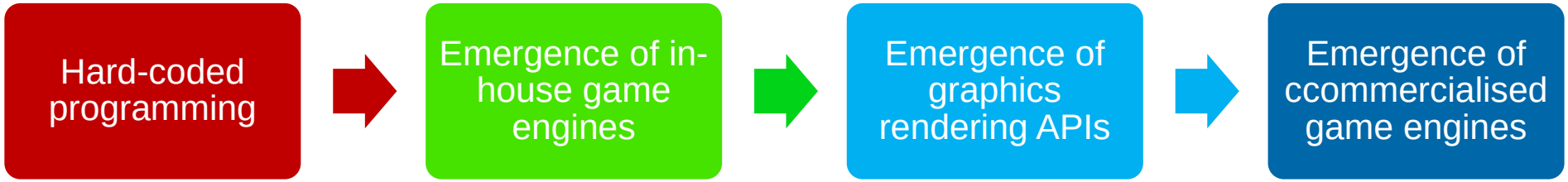
- Almost no one creates their own engine any more
- In 2021 so far, fewer than 20% of games launched have been created with "unknown" engines (in-house engines).



https://www.gamedeveloper.com/business/game-engines-on-steam-the-definitive-breakdown#close-modal

Before game engines

- Hard coding from the scratch.
- In-house game engines in the mid-1980s.
- Rendering application programming interfaces (API) such as OpenGL for rendering 2D and 3D graphics in 1992.
- Commercialised game engines, such as Quake engine or Unreal engine in the early 2000s.



Should you learn OpenGL or Game Engine?

- You need to learn OpenGL if you want to develop your own game engines or graphics libraries.
- Advantages of OpenGL:
  - You know what happens at the lowest level.
  - You have a good grasp of how the various game engines and libraries have been built.
- Disadvantages of OpenGL:
  - OpenGL is just a graphic API. You'll have to write your own libraries for Audio, Physics, collision detection, materials, networking etc.
  - It is not practical to develop a high-level game using OpenGL.
- Advantages of Game Engines:
  - Saves you a lot of time and helps you focus more on your game design and development than its backend.
  - Significantly easier to develop games compared to OpenGL.

Difference between rendering API's and game engines

- Rendering APIs like OpenGL or DirectX provide just functionality to abstract a graphics accelerator, focusing on rendering primitives, state management, command lists/command buffers.
- They are different from fully fledged 3D graphics libraries, 3D game engines (which handle scene graphs, lights, animation, materials etc.).

Does Unreal engine use OpenGL or DirectX?

- Unreal has an abstraction layer graphics rendering API (called RHI) above platform-dependent APIs such as DirectX 11, Vulkan, and OpenGL .
- Unreal engine uses DirectX 11 and Vulkan on Windows, OpenGL on Linux, and Metal on Mac.

Does Unity engine use OpenGL or DirectX?

- Unity uses its own built-in graphics rendering APIs by default.
- The default graphics API can be changed to DirectX 11 and Vulkan for Windows, OpenGL for Mac and Linux.

Why will you learn game engine rather than graphics APIs in this course?

- Graphics APIs are used for developing fundamental elements of computer graphics such as Graphic card drivers, game engines and rendering systems.
- Learning graphics APIs without knowing what a game engine doesn't have a lot of real world implication.
- Learning graphics API’s after learning game engines, can help you to know how to use graphic libraries in practice to improve the current game engines and rendering systems.

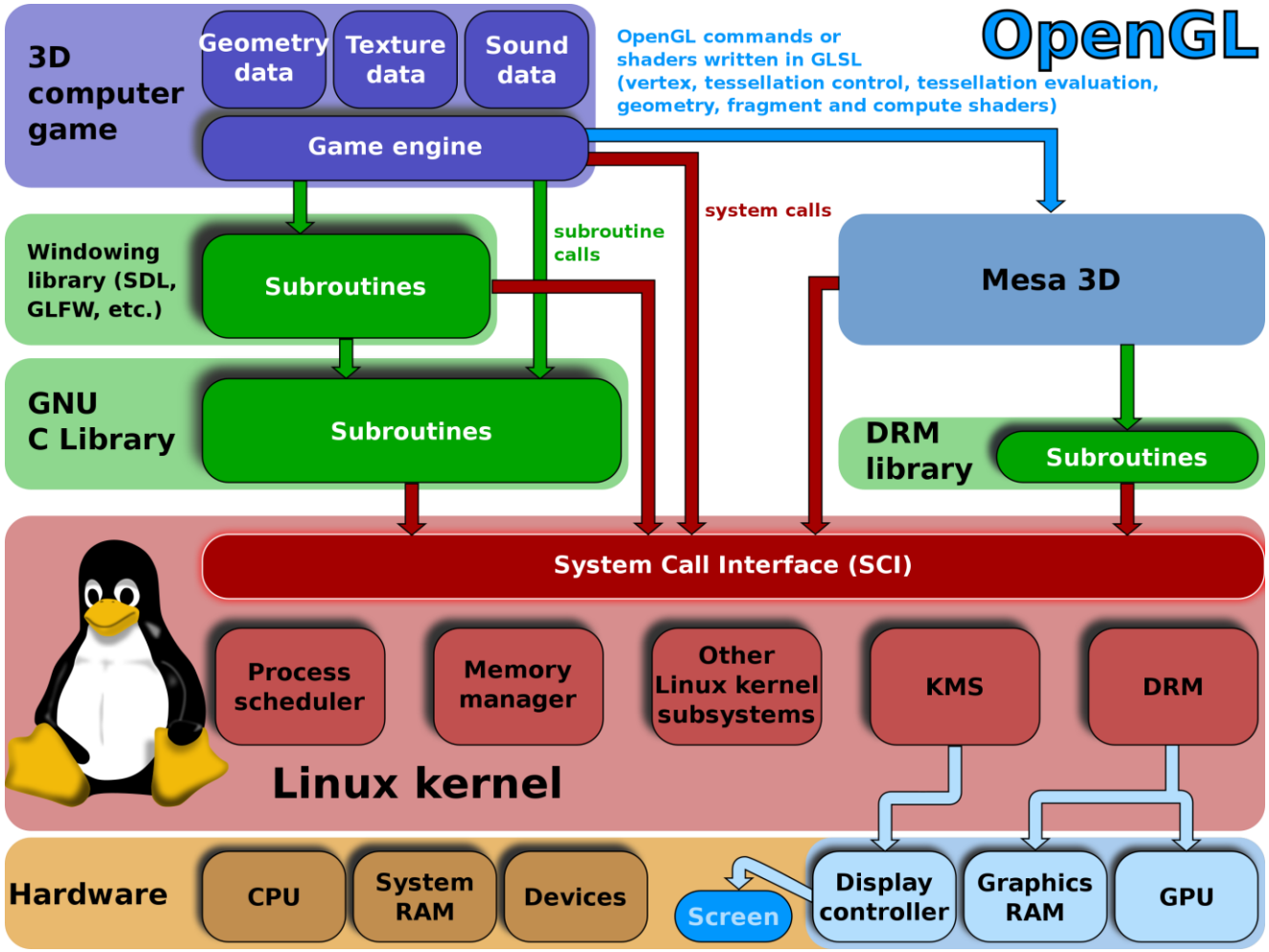
What is OpenGL (Open Graphics Library) rendering API?

- OpenGL is a cross-platform graphics application programming interface (API) for rendering 2D and 3D vector graphics.
- OpenGL is typically used to interact with a graphics processing unit (GPU), to achieve hardware-accelerated rendering.
- Since OpenGL is a graphics API and not a platform of its own, it requires a language to operate in and the language of choice is C++ or JavaScript.

Graphic card and openGL

- The people developing the actual OpenGL libraries are usually the graphics card manufacturers.
- Whenever there is a bug in the Graphic card function, this is solved by updating your video card drivers to get any version of OpenGL.

OpenGL and video games

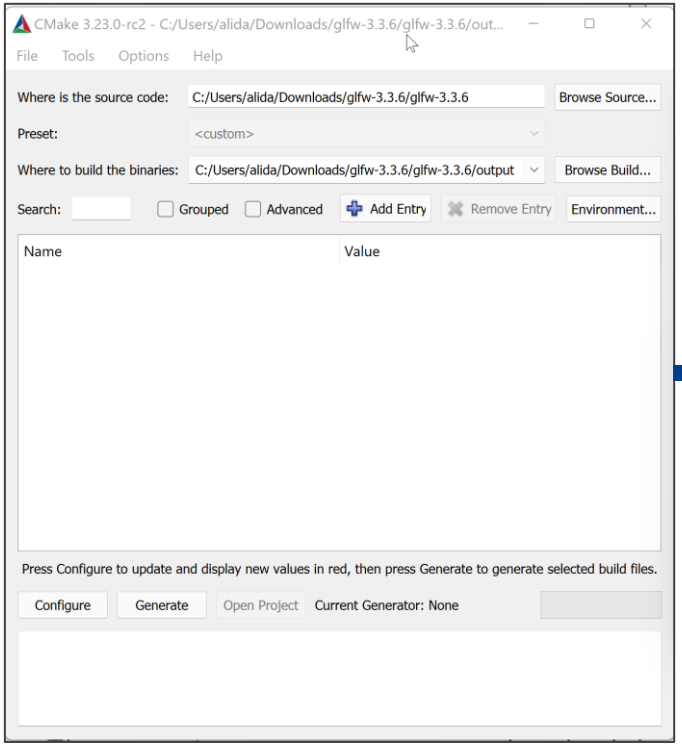


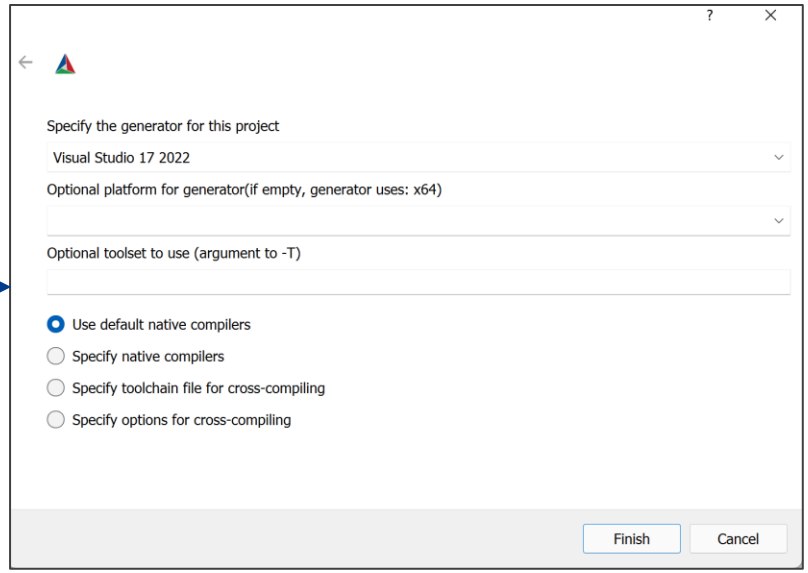
OpenGL libraries

- Toolkits to create and manage OpenGL windows, and manage input:
  - **GLFW** – A cross-platform game-oriented windowing and keyboard, mouse, and joystick handler.
  - **Freeglut** – A cross-platform windowing and keyboard-mouse handler.
- Multimedia libraries to create OpenGL windows, in addition to input, sound and other tasks useful for game-like applications:
  - **Allegro 5** – A cross-platform multimedia library with a C API focused on game development.
  - **Simple DirectMedia Layer (SDL)** – A cross-platform multimedia library with a C API.
  - **SFML** – A cross-platform multimedia library with a C++ API and multiple other bindings to languages such as C#, Java, Haskell, and Go.

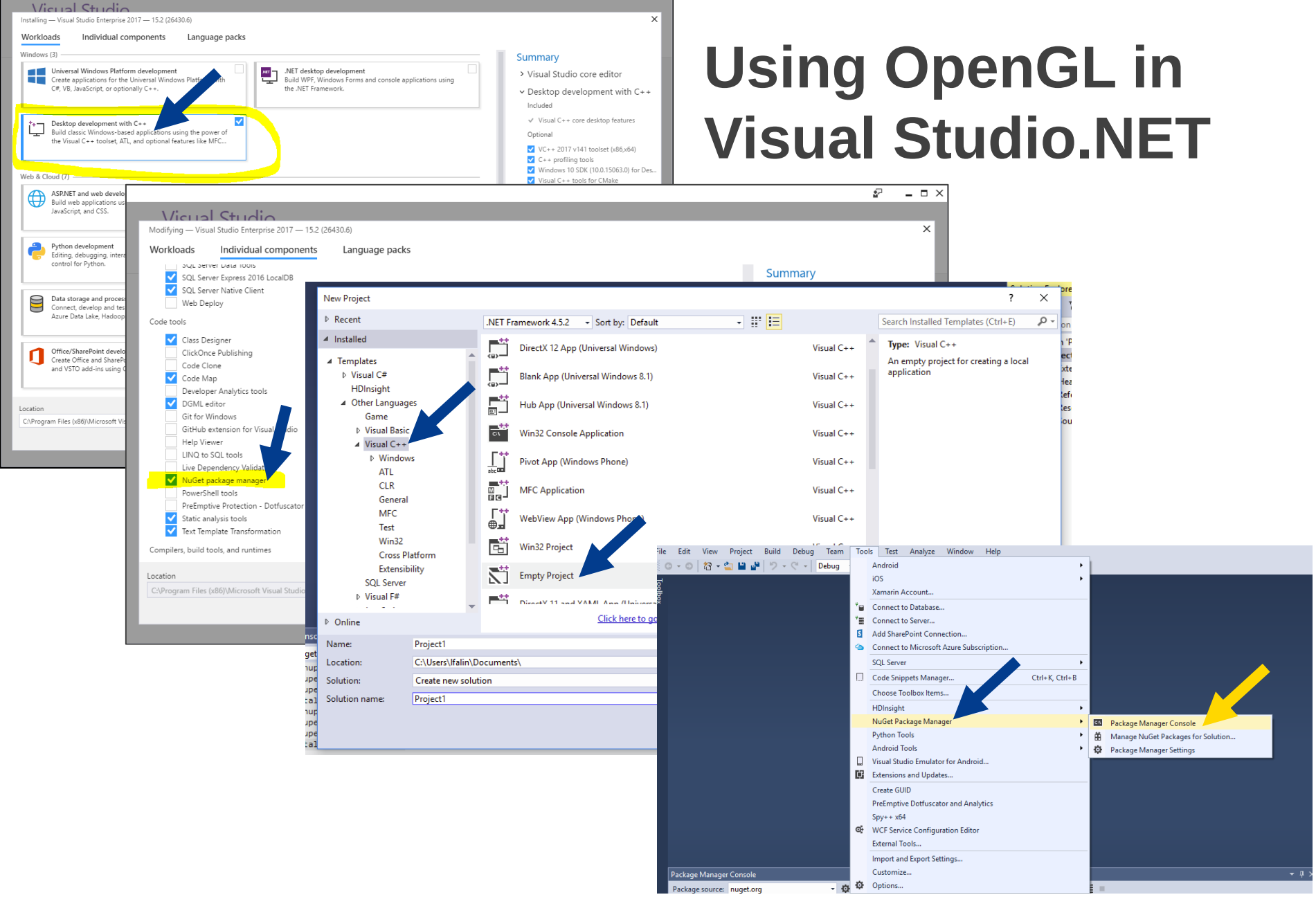
GLFW library

- Go to: https://www.glfw.org/download.html
- Download CMaker: https://cmake.org/download/





Using OpenGL in Visual Studio.NET

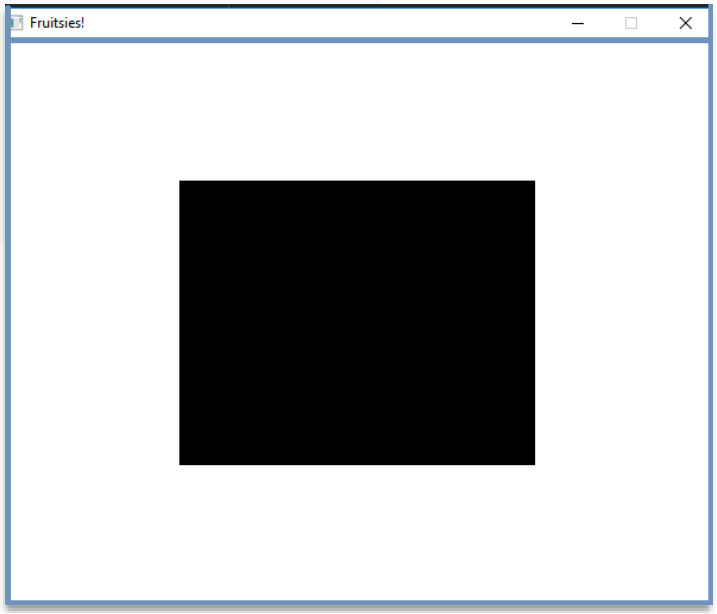


UNSW AUSTRALIA

Drawing a black rectangle in OpenGL

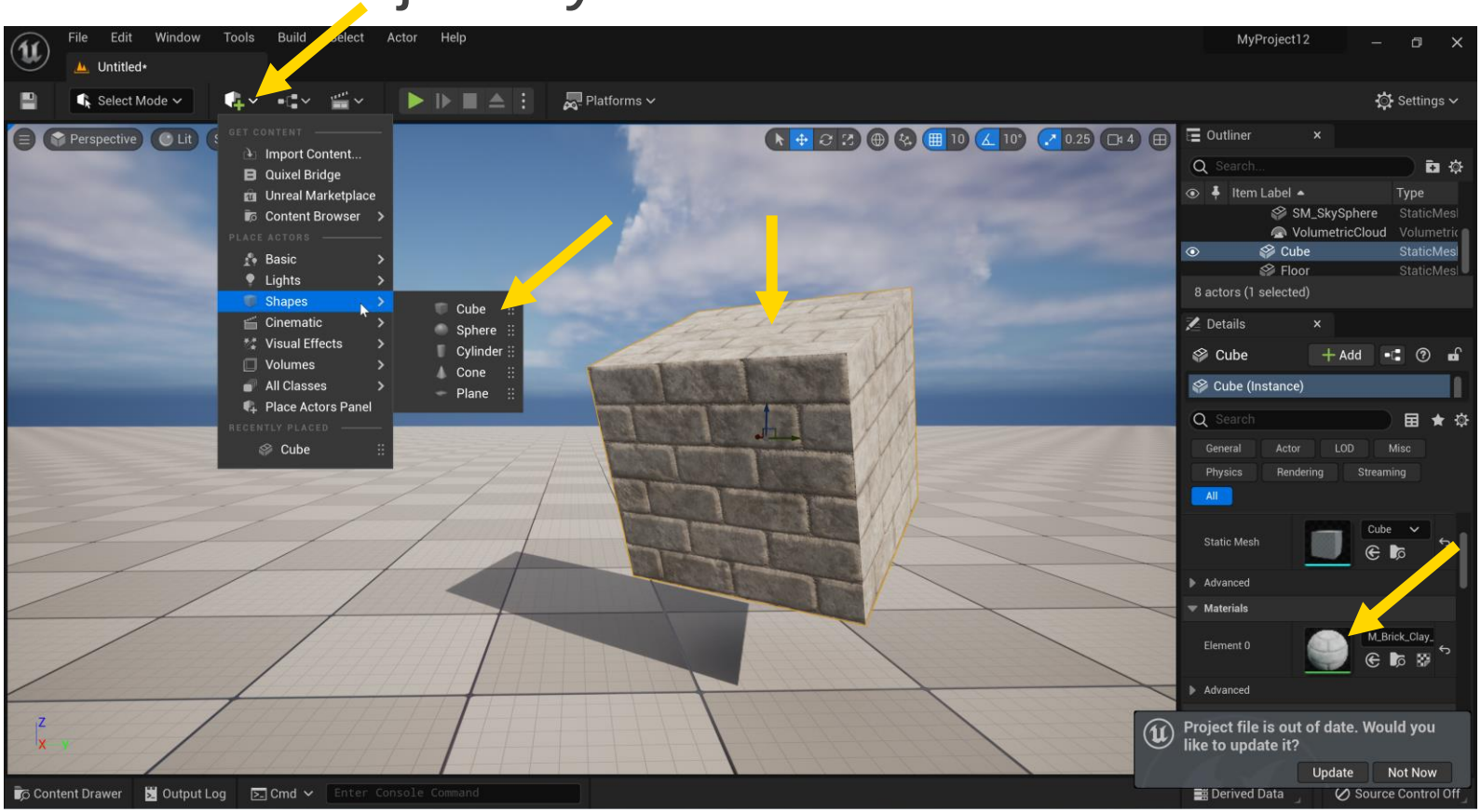
- The glClear() provides a clean slate to work on.
- The glColor3f function sets the current color.
- The glRectf function draws a rectangle.

```
void display() {
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        glColor3f(0.0f, 0.0f, 0.0f);
        glRectf(-0.75f,0.75f, 0.75f, -0.75f);
        glutSwapBuffers();
}
```



Drawing a rectangle in Unreal engine 5

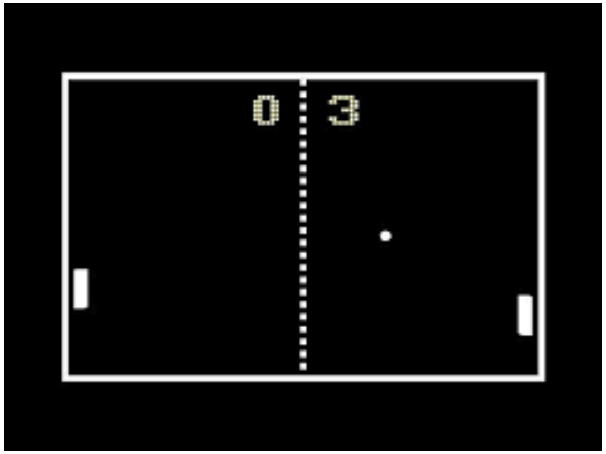
- Drawing a 3D rectangle (Cube) with brick material and shadow just by 5 clicks.



History of video games and game engines

First generation games (1960 - 1970):

- Very simple games
- Very Low graphics (black and white)
- Everything was designed from the scratch
- Game consoles with No CPU and graphic card
- Hard coded



History of video games and game engines

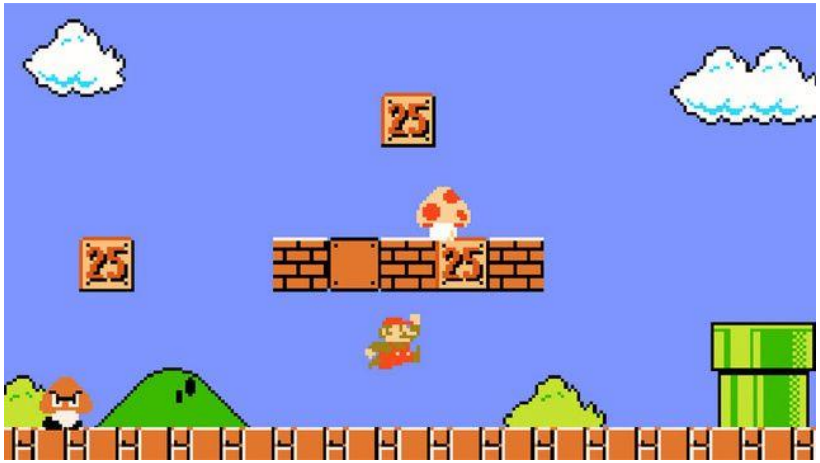
Second generation games (1970-1980):

- Very simple games (2KB cartridges)
- Very Low graphics (Color)
- Game consoles with CPU = 8-bit, 1–2 MHz and no graphic card
- Hard coded

History of video games and game engines

Third generation games (1980-1990):

- Simple games
- Low graphics (128 KB to 384 KB)
- Game consoles with CPU = 8-bit, 2–4 MHz and no graphic card
- Hard coded and in-house game engines



History of video games and game engines

Forth generation games (1990-1995):

- Simple games
- Medium resolution graphics (1MB to 5MB)
- Game consoles with CPU = 8-bit and 16-bit, 4–8 MHz and no graphic card
- Hard coded and in-house game engines

History of video games and game engines

Fifth generation games (1995-2000):

- Simple games
- Medium resolution graphics (5MB to 650MB)
- Game consoles with CPU = 32 and 64-bit, 12–100 MHz and no graphic card
- Graphic rendering API‘s and In-house game engines

History of video games and game engines

6th – 9th generation games (2000-present):

- Complex games (Nonlinear / Open world)
- High resolution graphics (1GB to 100GB)
- Developed by Game engines

	CPU	Graphic card
Sixth	32-bit, 200–733 MHz	100–233 MHz
Seventh	32-bit, 729 MHz–3.3 GHz	243–550 MHz
Eighth	32 and 64-bit, 1.0–2.3 GHz	300–1172 MHz
Ninth	64-bit, 3.4–3.8 GHz	1565–2233 MHz

History of video games and game engines

- A sample video game (Matrix) with Unreal Engine 5 on Play Station 5.

Course Evaluation and Development

- In response to feedback from previous offerings of this course, we've shifted the syllabus away from OpenGL and GLSL.
- The new focus is on game engines that are relevant in the industry, such as Unreal Engine 5.1.
- All lecture slides for this term have been updated based on Unreal Engine 5 to keep you up-to-date.
- Two Options for Phase Two of the Project.
- Recognizing that not all students have access to a VR headset, and acknowledging that the lab's VR headsets might not be sufficient for everyone, we are offering an alternative.

FAQ related to the course

Q1: Is it advisable to install Unreal Engine or Unity on our laptops for the tutorials? Should we also bring headsets if we have them?

A: Installing Unreal Engine or Unity on your laptop is not mandatory, but you're welcome to use your own laptop and headset for the tutorials.

Q2. Can game engines work on MacBook Pro?

A: Yes.

Q3: Can we expand the scope of our project beyond the content that we learn in this course?

A. Yes.

FAQ related to the course

Q4: Where should we store our projects?

A: You should store your projects on your UNSW OneDrive account.

Q5: Can we save our files on the VR lab computers?

A: Whenever possible, try to use the same computer in the VR lab. Always back up your files to cloud storage for added security.

Q6: Is it possible to use the lab computers during non-tutorial hours, such as in the evenings or on weekends?

A: Yes.